

■ 2.2.17 Advanced Topic: Reducing Expressions to Standard Form

The fact that addition is *associative* means that you can insert parentheses anywhere in a sum of terms, without changing the value of the sum. As a result, expressions like $(a + b) + c$, $a + (b + c)$ and $a + b + c$, that have different parenthesization, are all mathematically equal. In many senses, however, the most “natural” form for all these expressions is the unparenthesized one $a + b + c$. You can think of this unparenthesized form as being the “standard” or “canonical” form for the sum of terms.

If you enter a sum of terms into *Mathematica*, it will always try to remove parentheses so as to reduce the sum to the standard form.

Mathematica removes the parentheses to give this sum in the standard, unparenthesized, form.

```
In[1]:= (a + b) + c
Out[1]= a + b + c
```

Whenever *Mathematica* knows that a function is associative, it tries to remove parentheses (or nested invocations of the function) to get the function into a standard “flattened” form.

A function like addition also has the property of being commutative, which means that expressions like $a + c + b$ and $a + b + c$ with terms in different orders must be equal. Once again, *Mathematica* tries to put all such expressions into a “standard” form. The standard form it chooses is the one in which all the terms are in a definite order, corresponding roughly to alphabetical order.

Mathematica sorts the terms in this sum into a standard order.

```
In[2]:= c + a + b
Out[2]= a + b + c
```

flat (associative)	$f(f(a), b)$, etc. are equivalent to $f(a, b)$
orderless (commutative)	$f(b, c, a)$, etc. are equivalent to $f(a, b, c)$

Two important properties that *Mathematica* uses in reducing functions to standard form.

There are several reasons to try and put expressions into standard forms. The most important is that if two expressions are really in standard form, it is obvious whether they are equal or not.

When the two sums are put into standard order, they are immediately seen to be equal, so that two *f*’s cancel, leaving the result 0.

```
In[3]:= f[a + c + b] - f[c + a + b]
Out[3]= 0
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

You could imagine finding out whether $a + c + b$ was equal to $c + a + b$ by testing all possible orderings of each sum. It is clear that simply reducing both sums to standard form is a much more efficient procedure.

One might think that *Mathematica* should somehow automatically reduce *all* mathematical expressions to a single standard canonical form. With all but the simplest kinds of expressions, however, it is quite easy to see that you do not want the *same* standard form for all purposes.

For polynomials, for example, there are two obvious standard forms, which are good for different purposes. The first standard form for a polynomial is a simple sum of terms, as would be generated in *Mathematica* by applying the function **Expand**. This standard form is most appropriate if you need to add and subtract polynomials.

There is, however, another possible standard form that you can use for polynomials. By applying **Factor**, you can write any polynomial as a product of irreducible factors. This canonical form is useful if you want to do operations like division.

Expanded and factored form are in a sense both equally good standard forms for polynomials. Which one you decide to use simply depends on what you want to use them for. As a result, *Mathematica* does not automatically put polynomials into one of these two forms. Instead, it gives you functions like **Expand** and **Factor** that allow you explicitly to put polynomials in whatever form you want.

Here is a list of two polynomials that are mathematically equal.

```
In[4]:= t = {x^2 - 1, (x + 1)(x - 1)}
Out[4]= {-1 + x^2, (-1 + x)(1 + x)}
```

You can write both of them in expanded form just by applying **Expand**. In this form, the equality of the polynomials is obvious.

```
In[5]:= Expand[t]
Out[5]= {-1 + x^2, -1 + x^2}
```

You can also see that the polynomials are equal by writing them both in factored form.

```
In[6]:= Factor[t]
Out[6]= {(-1 + x)(1 + x), (-1 + x)(1 + x)}
```

Although it is clear that you do not always want expressions reduced to the *same* standard form, you may wonder whether it is at least *possible* to reduce all expressions to *some* standard form.

There is a basic result in the mathematical theory of computation which shows that it is, in fact, not always possible. You cannot guarantee that any finite sequence of transformations will take any two arbitrarily chosen expressions to a standard form.

In a sense, this is not particularly surprising. If you could in fact reduce all mathematical expressions to a standard form, then it would be quite easy to tell whether any two expressions were equal. The fact that so many of the difficult problems of mathematics can be stated as questions about the equality of expressions suggests that this is not always easy.